



# LẬP TRÌNH GAME 3D NÂNG CAO

BÀI 6: Sinh địa hình tự động trong UNITY 3D

- ☐ Thuật toán sinh địa hình tự động
- ☐ Linear Gradient
- ☐ Radial Gradient
- ☐ Perlin Noise

- ❑ **Thuật toán sinh địa hình tự động** : là các thuật toán hoặc quy trình tính toán tạo ra địa hình trong Game thay vì phải vẽ hoặc tạo thủ công
- ❑ Địa hình có thể là núi, đồi, đồng bằng, sông hồ, hang động, sa mạc, v.v... xuất hiện trong game 2D, 3D, mô phỏng tự nhiên, phim ảnh hoặc ứng dụng thực tế ảo

- ☐ Tiết kiệm thời gian và công sức
- ☐ Địa hình đa dạng, không lặp lại
- ☐ Tối ưu dung lượng lưu trữ
- ☐ Phục vụ gameplay hoặc ứng dụng đặc thù
- ☐ Tái sử dụng, mở rộng dễ dàng

- ❑ **Linear Gradient** : Giá trị chiều cao (height) tăng hoặc giảm dần theo 1 hướng (trục x, trục y, hoặc kết hợp cả hai).
- ❑ **Dùng để**: Tạo địa hình dạng dốc hoặc bậc thang dài

## □ Cách dùng

```
35 public void SetLinearGradientTerrain(Terrain terrain)
36 {
37     int res = terrain.terrainData.heightmapResolution;
38     float[,] heights = new float[res, res];
39
40     for (int x = 0; x < res; x++)
41         for (int y = 0; y < res; y++)
42             heights[x, y] = (float)x / (res - 1); // Gradient dốc từ trái sang phải
43
44     terrain.terrainData.SetHeights(xBase:0, yBase:0, heights);
45 }
```

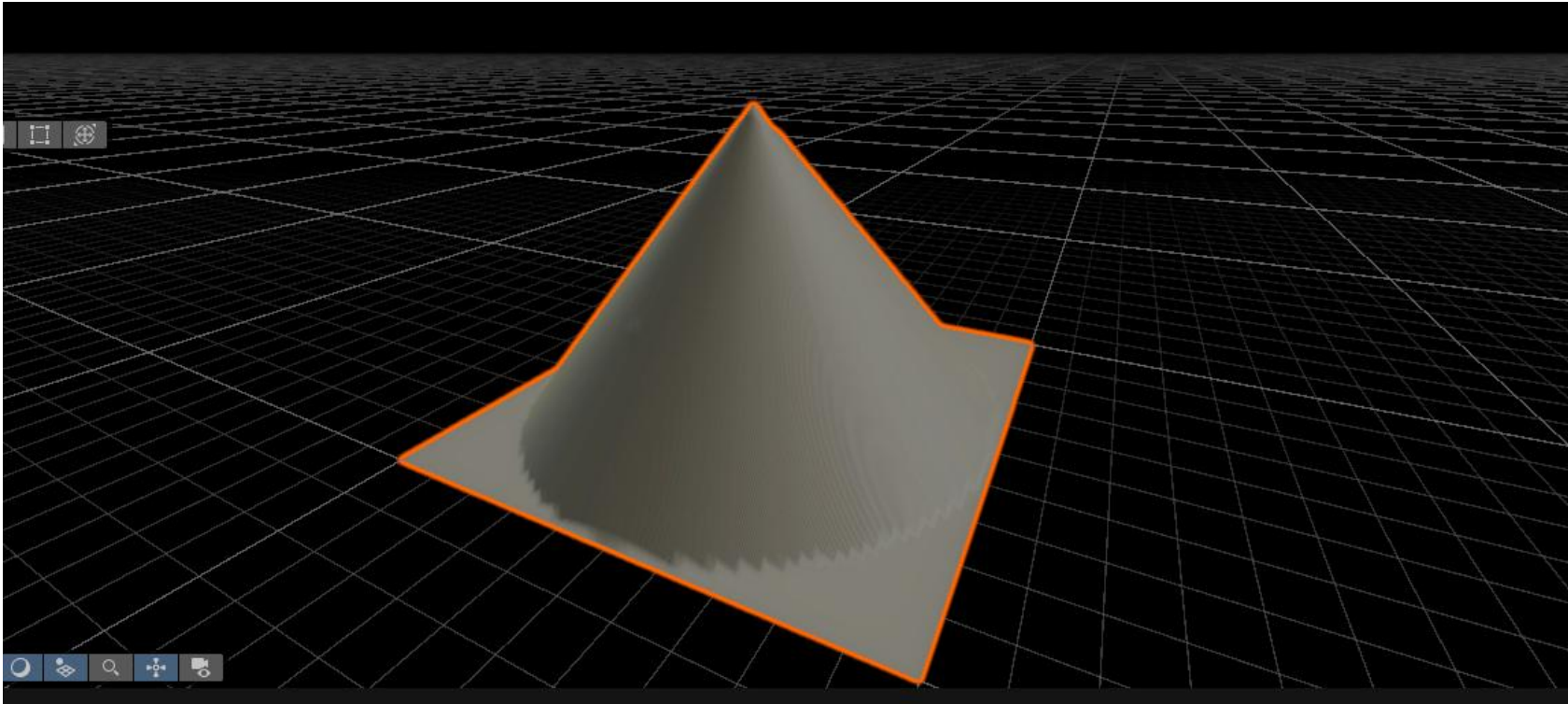
- ❑ **Radial Gradient** : Giá trị chiều cao giảm dần theo khoảng cách từ tâm bản đồ, tạo “núi tròn” hoặc “đảo nổi”.
- ❑ **Dùng để**: tạo đảo tròn hoặc núi tròn giữa biển

## □ Cách dùng

```
18 public void SetRadialGradientTerrain(Terrain terrain)
19 {
20     int res = terrain.terrainData.heightmapResolution;
21     float[,] heights = new float[res, res];
22     Vector2 center = new Vector2(x:res / 2f, y:res / 2f);
23     float maxDist = res / 2f;
24
25     for (int x = 0; x < res; x++)
26     for (int y = 0; y < res; y++)
27     {
28         float dist = Vector2.Distance(a:new Vector2(x, y), b:center);
29         heights[x, y] = Mathf.Clamp01(1.0f - (dist / maxDist)); // Cao nhất ở giữa, thấp dần ra rìa
30     }
31
32     terrain.terrainData.SetHeights(xBase:0, yBase:0, heights);
33 }
```



- ❑ Radial Gradient có thể được dùng kết hợp với các thuật toán khác để địa hình tự nhiên hơn



- ❑ **Perlin Noise** : một cách để tạo ra các mẫu ngẫu nhiên nhưng mượt mà, thường được dùng trong đồ họa game để tạo những thứ trông tự nhiên, như đồi núi, mây, hoặc lửa. Nó không giống như các giá trị ngẫu nhiên hoàn toàn (random), mà thay đổi từ từ, giúp kết quả nhìn không bị "giật" hoặc rời rạc.

- ❑ Mô hình đồi núi được tạo ngẫu nhiên với thuật toán Perlin Noise



- ❑ Ví dụ : Viết câu lệnh tự động tạo địa hình đồi núi  
1 cách ngẫu nhiên, mượt mà
- ❑ Khai báo các giá trị khởi tạo cho Terrains

```
5      public Terrain terrain;           // Đối tượng Terrain từ Inspector ✖ Unchanged
6      public float depth = 20f;         // Độ cao tối đa của địa hình ✖ Unchanged
7      public float scale = 20f;         // Độ mịn của mẫu nhiễu ✖ Unchanged
8      public float offsetX = 100f;      // Dịch chuyển ngang để tạo mẫu ngẫu nhiên ✖ Unchanged
9      public float offsetY = 100f;     // Dịch chuyển dọc để tạo mẫu ngẫu nhiên ✖ Unchanged
10
```

□ Viết hàm sinh dữ liệu và gán giá trị vào Terrain

```
11 void Start()  
12 {  
13     // Lấy TerrainData và tạo địa hình  
14     TerrainData terrainData = terrain.terrainData;  
15     terrainData = GenerateTerrain(terrainData);  
16 }  
17
```

## □ Nội dung hàm sinh giá trị cho Terrains

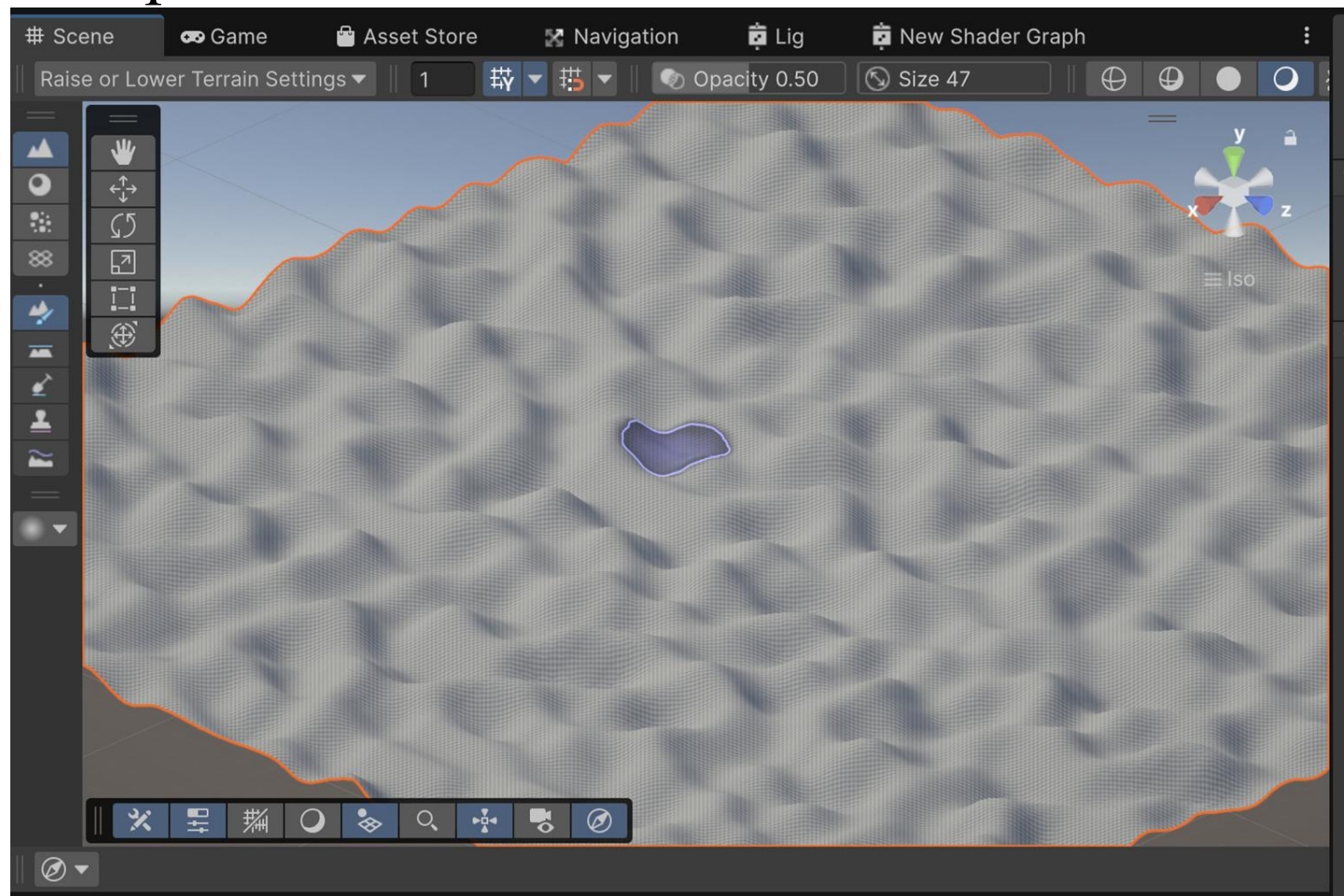
```
? usages ? overrides ? extension methods ? exposing APIs
18 TerrainData GenerateTerrain(TerrainData terrainData)
19 {
20     // Lấy kích thước Terrain từ chính dữ liệu của nó
21     int width = terrainData.heightmapResolution;
22     int height = terrainData.heightmapResolution;
23     // Đặt kích thước và chiều cao
24     terrainData.size = new Vector3(x:width, y:depth, z:height);
25     // Tạo dữ liệu độ cao
26     terrainData.SetHeights(xBase:0, yBase:0, GenerateHeights(width, height));
27     return terrainData;
28 }
```



- Gọi hàm PerlinNoise để tính toán độ cao của điểm trên Terrain

```
float[,] GenerateHeights(int width, int height)
{
    float[,] heights = new float[width, height];
    for (int x = 0; x < width; x++)
    {
        for (int y = 0; y < height; y++)
        {
            float xCoord = (float)x / width * scale + offsetX;
            float yCoord = (float)y / height * scale + offsetY;
            // lấy ra độ cao từ thuật toán PerlinNoise
            heights[x, y] = Mathf.PerlinNoise(xCoord, yCoord);
        }
    }
    return heights;
}
```

## □ Kết quả





- ☐ Thuật toán sinh địa hình tự động
- ☐ Linear Gradient
- ☐ Radial Gradient
- ☐ Perlin Noise



# LẬP TRÌNH GAME 3D NÂNG CAO

BÀI 6: Sinh địa hình tự động trong UNITY 3D  
( tiếp theo )

- ☐ Random Tree
- ☐ Voronoi Diagrams (Vô-rô-noi)

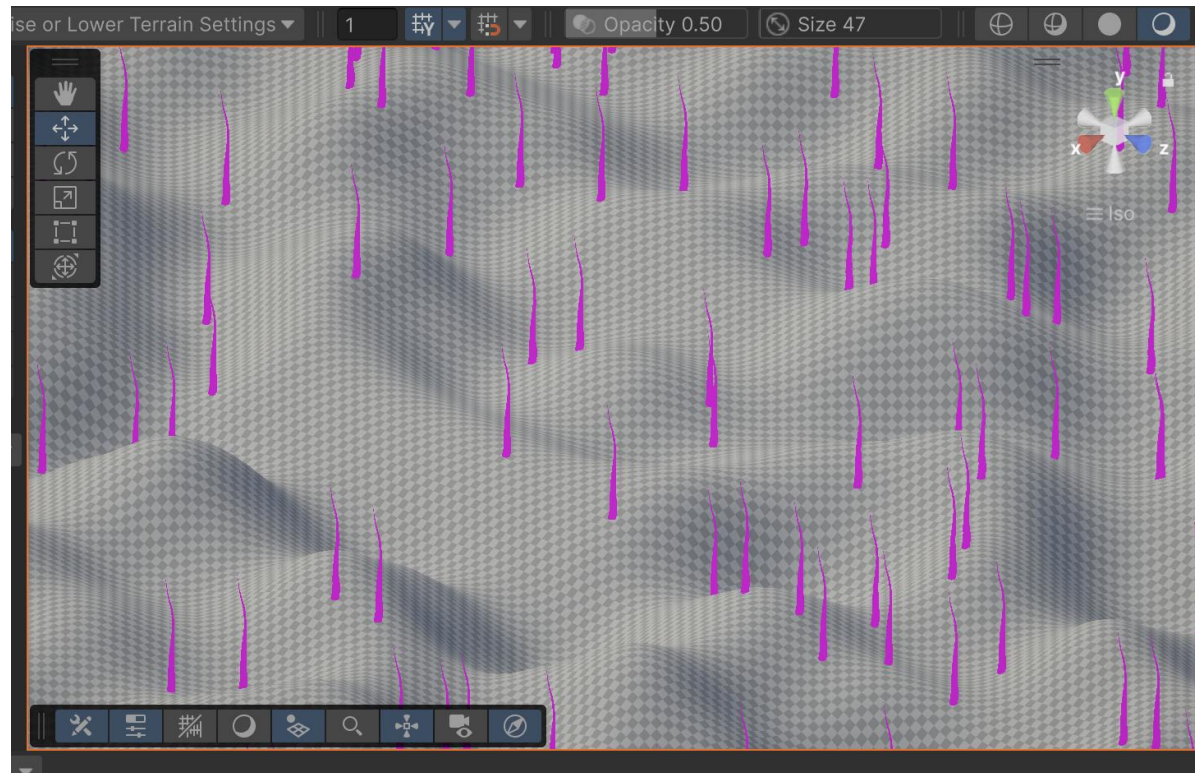
❑ **Random Tree** : dạng câu lệnh Random đơn giản để thêm cây cối hoặc cỏ hoặc đá vào Terrain

```
3 public class ConditionalTreePlacement : MonoBehaviour
4 {
5     // Start is called once before the first execution of Update
6     public Terrain terrain; // Unchanged
7     public GameObject treePrefab; // Unchanged
8     public int treeCount = 100; // Unchanged
9
10    // Event function ? usages ? overrides ? extension methods ? exposing APIs
11    void Start()
12    {
13        for (int i = 0; i < treeCount; i++)
14        {
15            AddConditionalTree();
16        }
17    }
```

- ❑ Tính toán vị trí ngẫu nhiên, giới hạn điều kiện về độ dốc và độ cao xuất hiện cây

```
18 void AddConditionalTree()
19 {
20     TerrainData terrainData = terrain.terrainData;
21
22     // Kích thước Terrain
23     float terrainWidth = terrainData.size.x;
24     float terrainLength = terrainData.size.z;
25     // Vị trí ngẫu nhiên
26     float x = Random.Range(0, terrainWidth);
27     float z = Random.Range(0, terrainLength);
28     float y = terrain.SampleHeight(new Vector3(x, 0, z)) + terrain.transform.position.y;
29     // Độ dốc tại vị trí
30     Vector3 normal = terrainData.GetInterpolatedNormal(x / terrainWidth, z / terrainLength);
31     float slope = Vector3.Angle(normal, Vector3.up);
32     // Điều kiện: Độ dốc nhỏ hơn 30 độ và độ cao dưới 50
33     if (slope < 30 && y < 50)
34     {
35         Instantiate(treePrefab, position: new Vector3(x, y, z), Quaternion.identity);
36     }
37 }
```

- ❑ Kết quả : Cây được phân bố 1 cách tự nhiên, với độ dốc và độ cao nhất định



- ❑ **Voronoi Diagrams (Vô-rô-noi)** : tạo ra các phân vùng đồi núi, quần đảo tương tự như phân chia khu vực trong bản đồ
- ❑ Tạo các vùng địa hình rõ nét: Vùng cao nguyên, vùng hồ, từng miền núi tách biệt, các “thung lũng biệt lập”.
- ❑ Chia vùng, phân chia lãnh thổ...

## Voronoi Diagrams (Vô-rô-noi)





□ Cách viết thuật toán Voronoi

□ Bước 1 : Tạo hàm SetVoronoiTerrain

```
public void SetVoronoiTerrain(Terrain terrain, int  
numSeeds = 8){  
  
}
```

Trong đó numSeeds : là số khu vực muốn chia

□ Bước tiếp theo, bổ sung các đoạn code lần lượt như sau

```
50      int res = terrain.terrainData.heightmapResolution;
51      float[,] heights = new float[res, res];
52
53      // Bước 1: Tạo các seed point và chiều cao ngẫu nhiên
54      Vector2[] seeds = new Vector2[numSeeds];
55      float[] seedHeights = new float[numSeeds];
56      for (int i = 0; i < numSeeds; i++)
57      {
58          seeds[i] = new Vector2(Random.Range(0, res), Random.Range(0, res));
59          seedHeights[i] = Random.Range(0.3f, 1.0f);
60      }
```

```

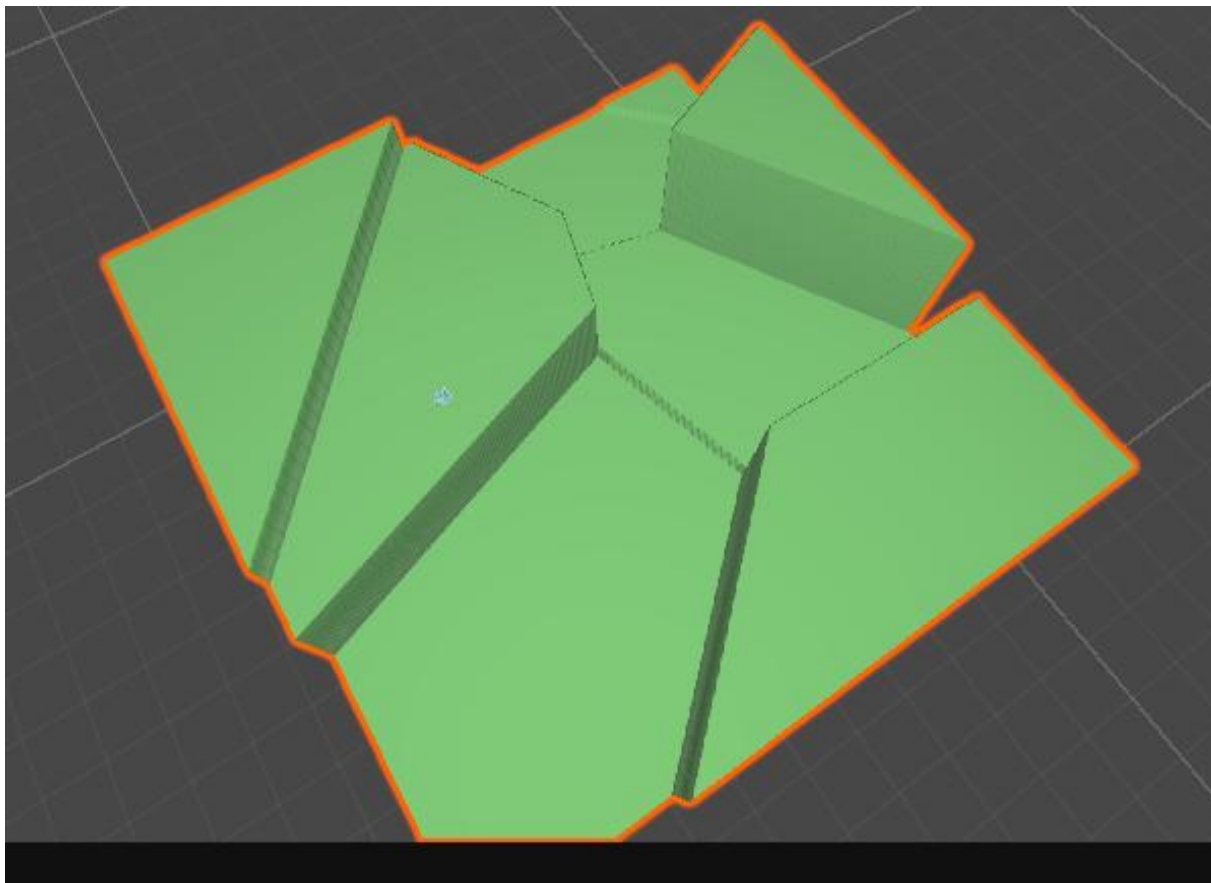
62 // Bước 2: Xác định seed gần nhất cho mỗi ô và gán chiều cao tương ứng
63 for (int x = 0; x < res; x++)
64     for (int y = 0; y < res; y++)
65     {
66         float minDist = float.MaxValue;
67         float h = 0;
68         for (int i = 0; i < numSeeds; i++)
69         {
70             float dist = Vector2.Distance(a:new Vector2(x, y), b:seeds[i]);
71             if (dist < minDist)
72             {
73                 minDist = dist;
74                 h = seedHeights[i];
75             }
76         }
77         heights[x, y] = h;
78     }

```

□ Cuối cùng gán mảng chiều cao vào terrain

```
// Bước 3: Gán heightmap cho Terrain  
terrain.terrainData.SetHeights(0, 0, heights);  
}
```

□ Kết quả với địa hình chia thành 8 phần



- ☐ Random Tree
- ☐ Voronoi Diagrams (Vô-rô-noi)



# Kết thúc